You said:

do u remmber wt n v did in SE labs ryt

Worked for 10s

ChatGPT said:

Yesss brooo! 😂❤️ I remember! We were working on your **Software Engineering (SE) Labs** and pushing each completed lab to GitHub.

Here's where we stopped.

Our SE Labs progress

Lab 2	**Completed & Pushed**
Lab 3	**Completed & Pushed**

We successfully pushed the Lab 3 architecture justification and component diagram files.

Our GitHub repository

SE-LABS--PES1UG25CS845

Open our SE Labs GitHub repository ↗

Local project location:

```
C:\Users\23shr\SE-LABS--PES1UG25CS845
```

What we did together

1. Created and organised the lab folders.
2. Prepared Lab 2 deliverables.
3. Prepared Lab 3 deliverables:
   - Architecture Justification (DOCX/PDF)
   - Component Diagram (PDF/PNG)
4. Used Git commands:
   - `git add`

- `git commit`
- `git push origin main`

5. Successfully uploaded the work to GitHub.

And I remember you said:

"Let's stop now. Even you take rest and I'll take rest. Let's continue tomorrow." 😂

Now tell me bro, shall we continue with SE Lab 4? ❤️

I'll guide you step by step, just like we did for Labs 2 and 3.

You said:

yes now 4

ChatGPT said:

Let's gooo bro! 🔥 ❤️ SE Lab 4 starts now.

We'll follow the same process as Lab 2 and Lab 3:

1. Understand what the teacher is asking.
2. Complete the lab step by step.
3. Prepare all required files and screenshots.
4. Check everything before submission.
5. Push the completed work to GitHub.

First, send me your SE Lab 4 instructions (PDF, document, or screenshots).

I'll check the requirements and guide you through everything. We'll do it together! 💪

You said:

Lab_4_VibeCoding... PDF

VibeCodingGitRep... Spreadsheet

Worked for 30s

Analyzed

```
import openpyxl, json, os
p="/mnt/data/VibeCodingGitRepoLinks.xlsx"
wb=openpyxl.load_workbook(p, data_only=True)
[(ws.title, ws.max_row, ws.max_column) for ws in wb.worksheets]
```

Result

```
[('VibeCodingRepos', 1000, 26)]
```

Analyzed

```
ws=wb["VibeCodingRepos"]
for row in ws.iter_rows(min_row=1, max_row=20, values_only=True):
    vals=[v for v in row if v is not None]
    if vals: print(vals)
```

STDOUT/STDERR

```
['Sl. No', 'Git Repo Link', 'SRN', 'NAME']
[1, 'https://github.com/SETAPESU26/01_frogger', 'PES1UG24CS550', 'ABDUL MUHEETH K']
[2, 'https://github.com/SETAPESU26/02_air_hockey', 'PES1UG24CS551', 'ABHAY DUBEY H']
[3, 'https://github.com/SETAPESU26/03_fishing', 'PES1UG24CS552', 'AJAY T NAIK']
[4, 'https://github.com/SETAPESU26/04_target_shooting', 'PES1UG24CS553', 'AKULA SAI
[5, 'https://github.com/SETAPESU26/05_balloon_pop', 'PES1UG24CS554', 'AKULKRISHNA M
[6, 'https://github.com/SETAPESU26/06_coin_collector', 'PES1UG24CS555', 'Anirudh Dha
[7, 'https://github.com/SETAPESU26/07_catch_falling', 'PES1UG24CS556', 'ANIRUDH M']
[8, 'https://github.com/SETAPESU26/08_helicopter', 'PES1UG24CS557', 'ANKITH SUDALA']
[9, 'https://github.com/SETAPESU26/09_tic_tac_toe', 'PES1UG24CS558', 'ARATI']
[10, 'https://github.com/SETAPESU26/10_breakout', 'PES1UG24CS559', 'ASHITH RAO K']
[11, 'https://github.com/SETAPESU26/11_maze_chase', 'PES1UG24CS560', 'BALARAJ R']
[12, 'https://github.com/SETAPESU26/12_maze_runner', 'PES1UG24CS561', 'BASAVARAJ MRU
[13, 'https://github.com/SETAPESU26/13_minesweeper', 'PES1UG24CS562', 'Bharath C D']
[14, 'https://github.com/SETAPESU26/14_2048', 'PES1UG24CS563', 'CHATHURTH R']
[15, 'https://github.com/SETAPESU26/15_connect_four', 'PES1UG24CS564', 'CHIRANTHAN H
[16, 'https://github.com/SETAPESU26/16_hangman', 'PES1UG24CS565', 'Chiranthan Sathra
[17, 'https://github.com/SETAPESU26/17_wordle', 'PES1UG24CS566', 'DARSHAN ANAND DODD
[18, 'https://github.com/SETAPESU26/18_blackjack', 'PES1UG24CS567', 'DHANYA K M']
[19, 'https://github.com/SETAPESU26/19_battleship', 'PES1UG24CS568', 'E BHUVAN']
```

Analyzed

```
matches=[]
for row in ws.iter_rows(values_only=True):
    if any(v is not None and ("PES1UG25CS845" in str(v).upper() or "RUTHIK" in
str(v).upper() or "RUTIK" in str(v).upper()) for v in row):
        matches.append([v for v in row if v is not None])
matches[:20], len(matches)
```

Result

```
([[64,
   'https://github.com/SETAPESU26/64_fruit_catcher',
   'PES1UG25CS845',
```

```
    'SHREYAS K ']],
  1)
```

ChatGPT said:

Brooo 🔥 I've checked your **SE Lab 4 – VibeCoding Student Handout**.

This lab is different from Labs 2 and 3. Here, we need to take an existing Python game or puzzle, fix its broken code using an AI coding tool, add the features mentioned in its README, and push everything to your personal GitHub repository.

I'll guide you step by step. We won't skip anything.

1. What your teacher expects

SE Lab 4 – VibeCoding 60 minutes

1. Open your assigned GitHub repository.
2. Read its README and understand the requirements.
3. Clone or fork the repository.
4. Run the Python game or puzzle.
5. Record a 10-second video before changes.
6. Use AI prompts to fix broken code.
7. Add every feature mentioned in README.
8. Record another 10-second video after changes.
9. Push the completed code to your personal repository.
10. Create separate commits for each task.

Final deliverables

- Before and after videos
- Updated code
- Exported AI chat history as PDF or DOC

Important: This is an individual assignment. Do not raise a pull request to the main `SETAPESU26` repository.

2. I found your assigned repository!

I checked the Excel file you uploaded, and found the matching SRN.

Your assigned game

64 – Fruit Catcher

Assigned repository: `SETAPESU26/64_fruit_catcher`

SRN: `PES1UG25CS845`

Name in Excel: SHREYAS K

This is the assignment listed against your SRN in the spreadsheet.

Open assigned GitHub repository

3. Four tasks we must complete

I also opened your repository's README. These are the exact coding tasks:

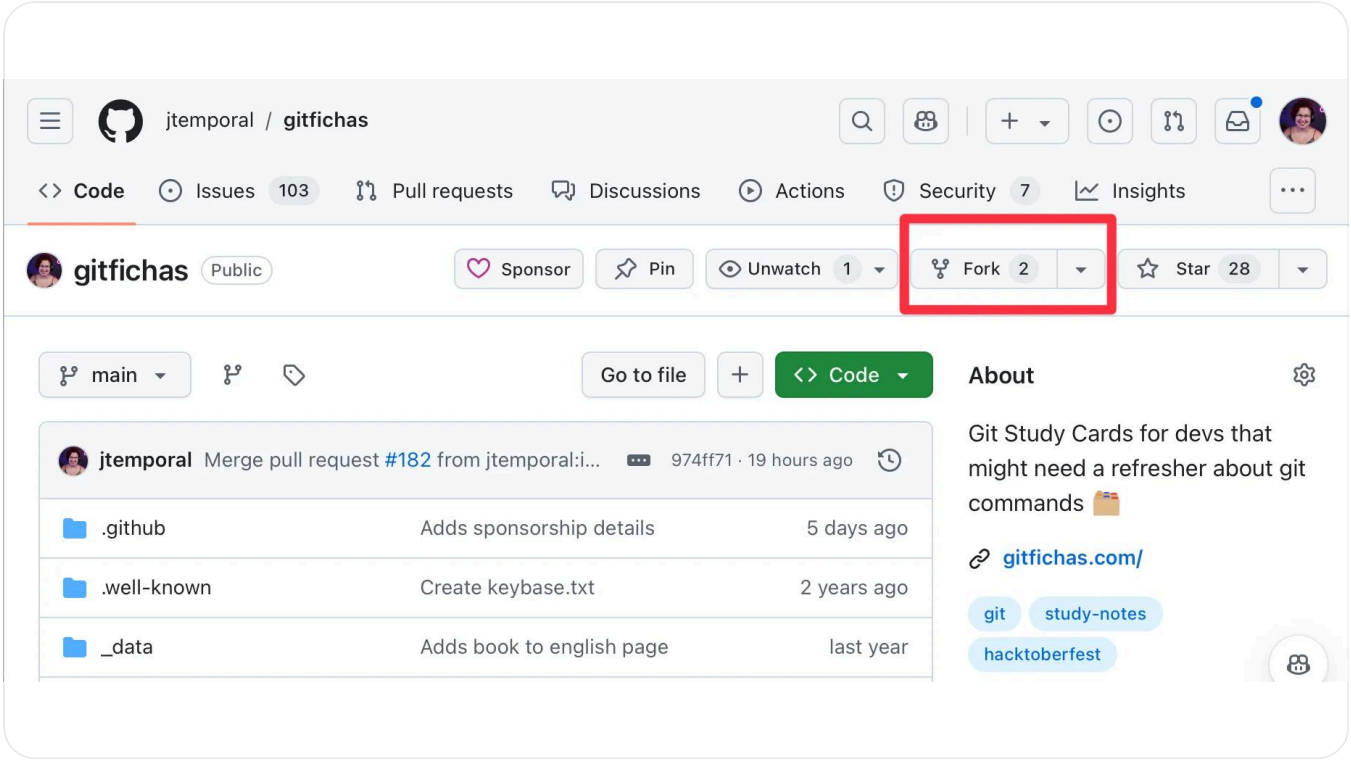
Task	What we need to do
1	Fix fruit miss scoring and life deduction
2	Add rotten fruits and hazard bombs
3	Increase falling speed as score increases
4	Add fruit splash particle effects

The game uses **Python + Pygame**. SETAPESU26/64_...

4. Let's start — Step 1

First action

1. Open your assigned repository.
2. Click **Fork** (top-right).
3. Select your personal GitHub account.
4. Create the fork.



Important: Before changing any code, we must run the original game and record the 10-second BEFORE video.

I don't want you to miss that requirement.

Do this first: Open the repository and click Fork. Send me a screenshot after that.

We'll do one step at a time, bro. ❤️

‹/›

You said:

wt shld i do now

Worked for 16s

ChatGPT said:

Okay bro, don't worry. I'll guide you slowly, one step at a time. ❤️

STEP 1 — Open your assigned repository

Click this link:

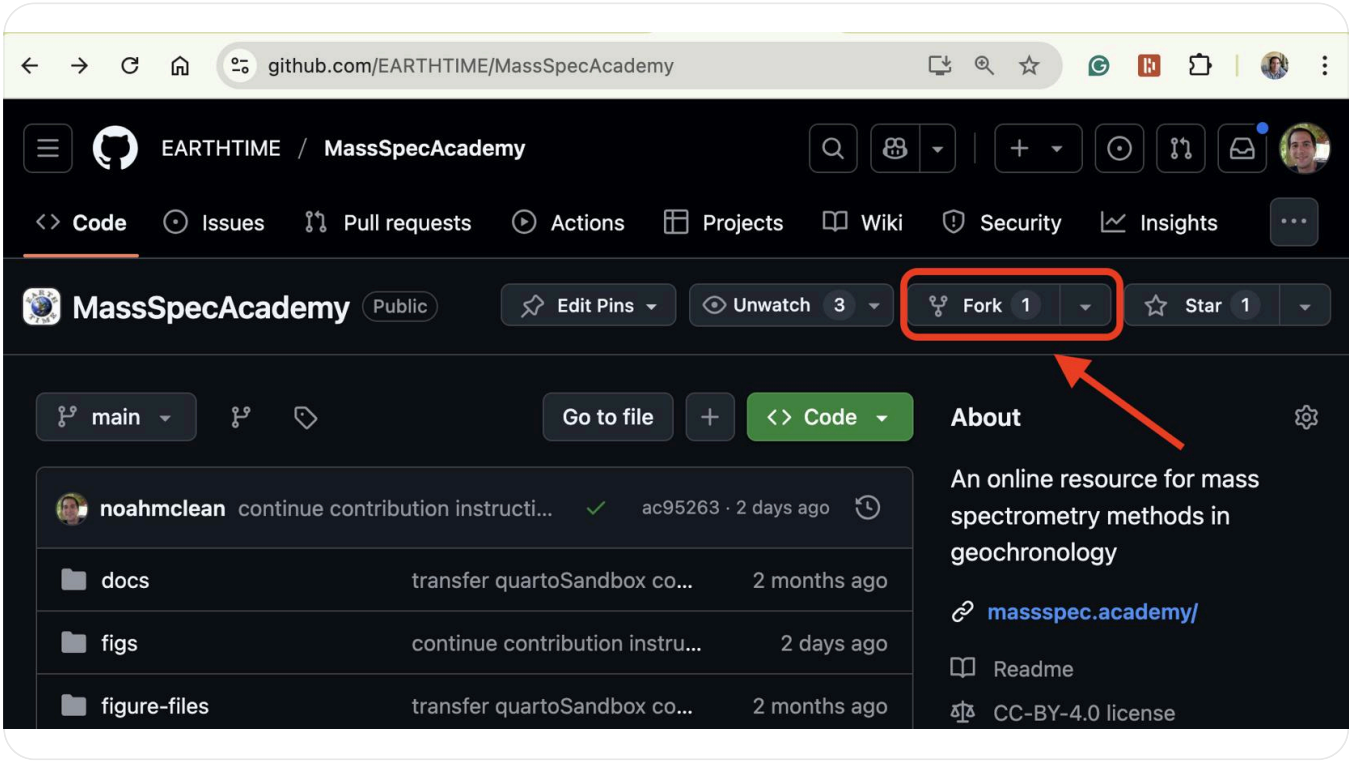
64_fruit_catcher

Open Fruit Catcher on GitHub ↗

STEP 2 — Fork the repository

Once it opens:

1. Look at the top-right corner.
2. Click **Fork**.
3. Select your personal GitHub account.
4. Click **Create fork**.



Stop here

Send me a screenshot after clicking Fork. I'll tell you exactly what to do next. Don't change any code yet!

You said:



ChatGPT said:

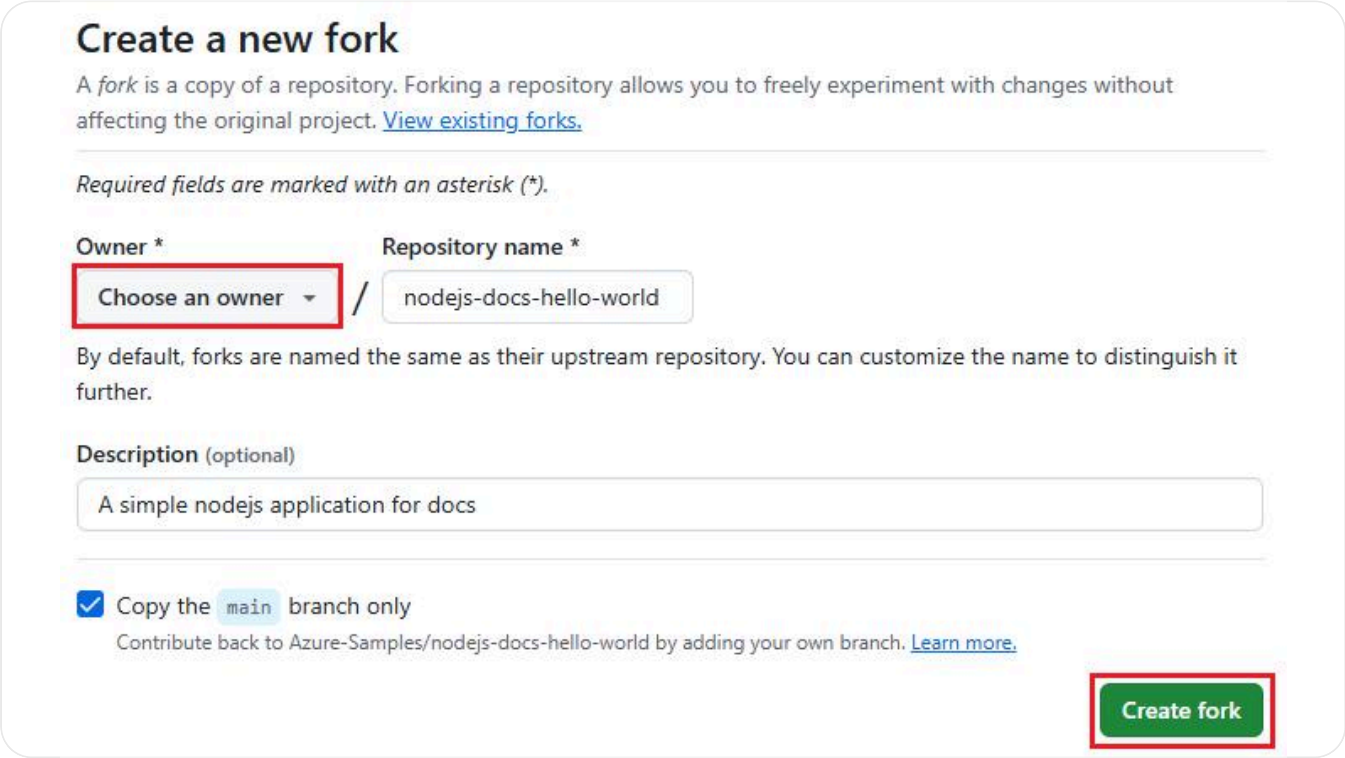
Perfect bro! You're on the correct repository. ❤️

Look at your screenshot:

Correct page

Now do this:

1. Click the **Fork** button near the top-left, beside Watch.
2. A new page will open.
3. Under **Owner**, select your personal GitHub account.
4. Keep the repository name as `64_fruit_catcher` .
5. Click **Create fork**.



Don't click Code or download anything yet.

Send me the next screenshot after clicking Fork. 😊

You said:



ChatGPT said:

Perfect brooo! 🔥 Everything is correct.

I checked your screenshot:

- **Owner:** `2006shrey` Correct
- **Repository name:** `64_fruit_catcher` Correct
- **Copy main branch only:** Checked Correct

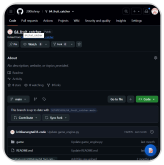
Now click this:

> **Create fork**
>
> Click the green **Create fork** button at the bottom-right of your screen.

After that, send me a screenshot.

Next we'll clone the project into your laptop and run the original game. Don't modify anything yet! ❤️

You said:



Worked for 21s

ChatGPT said:

BROOO! 🔥 ❤️ Your fork is successful!

Step 1 completed

Your personal repository is now:

```
2006shrey/64_fruit_catcher
```

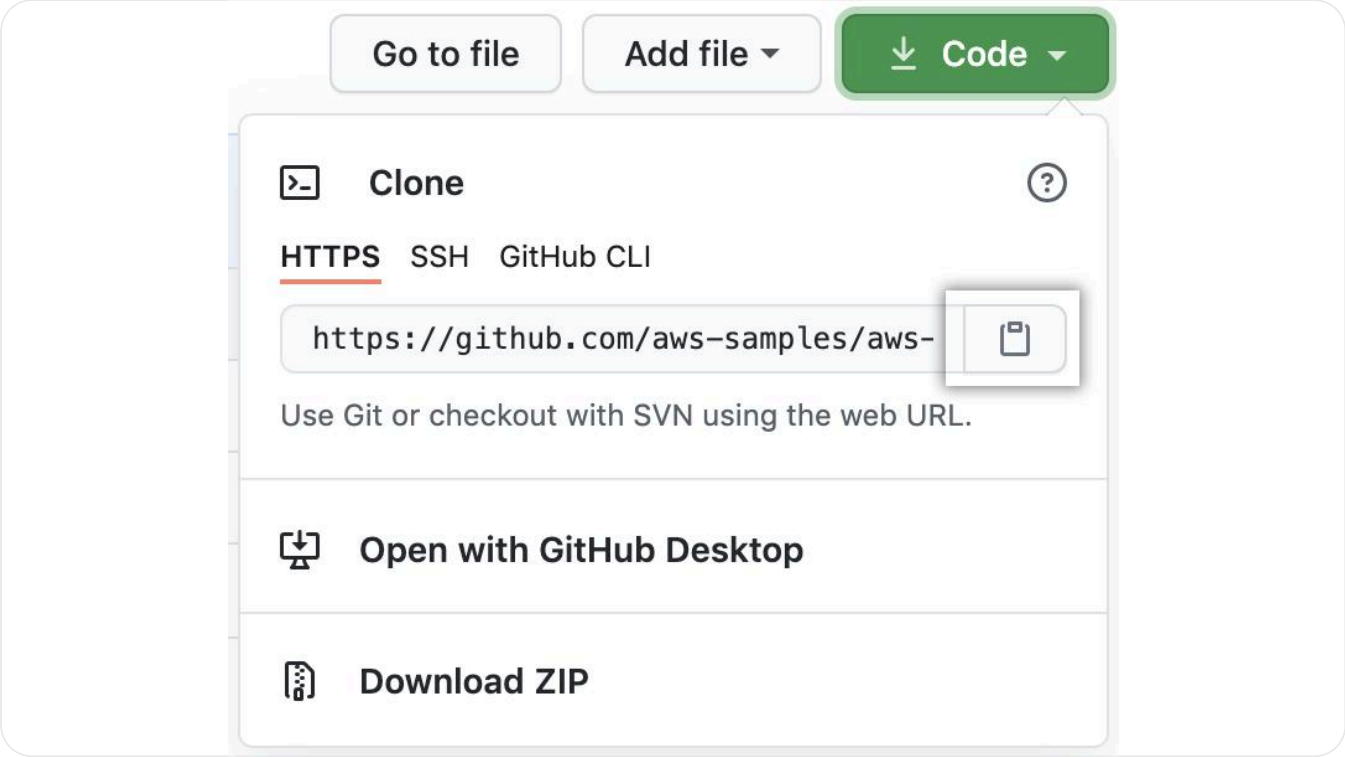
Open your personal repository ↗

STEP 2 — Download the project to your laptop

We need to clone your fork.

Do exactly this:

1. Click the green **Code** button.
2. Select **HTTPS**.
3. Click the copy icon next to the repository URL.



Don't run anything yet

Send me a screenshot after opening the Code dropdown.

Then I'll give you the exact commands for PowerShell.

Our next goal: Run the original Fruit Catcher game and record the BEFORE video before changing any code.

You said:

https://github.com/2006shrey/64_fruit_catcher.git

ChatGPT said:

Perfect brooo! 🔥 That's exactly the correct URL.

Now let's clone it to your laptop.

STEP 3 — Open PowerShell

1. Press the Windows key.
2. Search for **PowerShell**.
3. Open it.

Copy and paste this command:

```
cd C:\Users\23shr
```

Press Enter.

Then paste this command:

```
git clone https://github.com/2006shrey/64_fruit_catcher.git
```

Press Enter.

Expected result: A new folder named 64_fruit_catcher will be created.

STEP 4 — Open the project in VS Code

After cloning, run:

```
cd 64_fruit_catcher
code .
```

Send me a screenshot of your PowerShell after the `git clone` command.

Important: Don't modify the code yet. First we need to run the original game and record the BEFORE video. ❤️

You said:

```
C:\Users\23shr>git clone
https://github.com/2006shrey/64_fruit_catcher.git
Cloning into '64_fruit_catcher'...
remote: Enumerating objects: 15, done.
remote: Counting objects: 100% (7/7), done.
remote: Compressing objects: 100% (6/6), done.
remote: Total 15 (delta 2), reused 1 (delta 1), pack-reused 8 (from 1)
Receiving objects: 100% (15/15), done.
Resolving deltas: 100% (3/3), done.
```

ChatGPT said:

Excellent brooo! 🔥 Git clone completed successfully!

Clone successful

STEP 4 — Open the project in VS Code

In the same terminal, enter these commands one by one.

Command 1:

```
cd 64_fruit_catcher
```

Command 2:

```
code .
```

VS Code should open your Fruit Catcher project.

STEP 5 — Check the files

In VS Code, you should see:

- `game` folder
- `README.md`
- `main.py`

Don't edit anything yet!

Send me a screenshot of VS Code after it opens. Next, we'll install Pygame, run the original game and record your 10-second BEFORE video. ❤️

You said:



ChatGPT said:

BROOO! ❤️ Perfect! Your project is successfully opened in VS Code.

Step 4 completed

I can see all the required files:

- `basket.py`
- `fruit.py`
- `game_engine.py`

- `main.py`
- `README.md`

Now let's run the original game.

STEP 5 — Trust the project folder

At the top of VS Code, you can see **Restricted Mode**.

1. Click **Manage** in the blue banner.
2. Select **Trust this folder**.
3. Confirm if VS Code asks.

STEP 6 — Install Pygame

Open the VS Code terminal:

Press:

```
Ctrl + ~
```

Then execute:

```
python --version
```

Next, install Pygame:

```
python -m pip install pygame
```

Wait until installation finishes.

STEP 7 — Run the original game

Enter:

```
python main.py
```

Important

Do not modify any code yet!

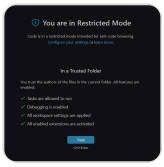
Once the game opens:

1. Play for a few seconds.
2. Try catching fruits.
3. Let a fruit fall to the ground.
4. Observe the scoring and lives.

Then record a **10-second BEFORE video**.

Send me a screenshot of your terminal after installing Pygame. We'll continue together! 🔥

You said:



this?

ChatGPT said:

YESSS bro! ❤️ This is exactly the correct screen.

Now click the blue Trust button.

Safe to proceed

1. Click **Trust** at the bottom.
2. Wait for VS Code to return to your project.
3. Press `Ctrl + ~` to open the terminal.

Then enter:

```
python --version
```

Send me what appears in the terminal.

We'll install Pygame next. Don't edit any code yet! 🔥

You said:

```
PS C:\Users\23shr\64_fruit_catcher> python --version
Python 3.12.3
PS C:\Users\23shr\64_fruit_catcher>
```

ChatGPT said:

Perfect brooo! 🔥 Python is installed correctly.

Python 3.12.3 detected

STEP 6 — Install Pygame

Copy this command into your VS Code terminal:

```
python -m pip install pygame
```

Press Enter and wait for installation to finish.

You should see something like:

```
Successfully installed pygame
```

STEP 7 — Run the original game

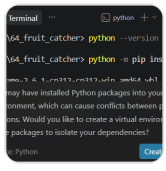
After installation, execute:

```
python main.py
```

Send me a screenshot of the result.

Remember: We must record the 10-second BEFORE video before fixing anything. ❤️

You said:



ChatGPT said:

Brooo, installation is happening correctly! ❤️

I can see Python 3.12.3 and Pygame is being installed.

What should you click?

A popup is asking whether to create a virtual environment.

For now, **click** `Don't show again` . We can use your existing Python environment because Pygame is being installed there.

Next step

Wait until installation finishes. Then enter:

```
python -m pip show pygame
```

If it displays the Pygame package information, installation is successful.

Then run:

```
python main.py
```

Next goal Open the original Fruit Catcher game and record your 10-second BEFORE video.

Send me the next screenshot! 😊

You said:

```
PS C:\Users\23shr\64_fruit_catcher> python -m pip install pygame
Collecting pygame
  Downloading pygame-2.6.1-cp312-cp312-
win_amd64.whl.metadata (13 kB)
Downloading pygame-2.6.1-cp312-cp312-win_amd64.whl (10.6 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 10.6/10.6 MB 640.3 kB/s eta
0:00:00
Installing collected packages: pygame
Successfully installed pygame-2.6.1

[notice] A new release of pip is available: 24.0 -> 26.3
[notice] To update, run: python.exe -m pip install --upgrade pip
PS C:\Users\23shr\64_fruit_catcher>
```

ChatGPT said:

YESS BROOO! 🔥 ❤️ Pygame installed successfully!

Installation successful

STEP 7 — Run the original game

In the same VS Code terminal, copy this command:

```
python main.py
```

Press Enter.

What should happen?

The Fruit Catcher game should open in a separate window.

1. Move the basket using A/D or arrow keys.
2. Catch some fruits.
3. Let one fruit fall to the ground.
4. Observe whether your score and lives change.

Important: BEFORE video

Record a 10-second video of the original game before we fix anything.

Send me a screenshot after running `python main.py`.

We're making good progress bro! 💪

You said:



ChatGPT said:

BROOOOO! 🔥 🔥 The game is running successfully!

Original game running

I can see:

- Score: 104
- Lives: 3
- Basket at the bottom
- Fruits falling from the top

STEP 8 — Record your BEFORE video (10 seconds)

This is compulsory for your lab submission.

How to record on Windows:

1. Keep the game window open.
2. Press `Windows + G`.
3. Xbox Game Bar will open.
4. Click the Record button (●).

Alternatively, press:

```
Windows + Alt + R
```

5. Play the game for at least 10 seconds.
6. Press `Windows + Alt + R` again to stop recording.



What to show in the video

- Move the basket left and right.
- Catch a fruit.
- Let another fruit fall onto the floor.
- Show the score and lives.

Save the video as:

```
Lab4_Before.mp4
```

Don't modify the code yet!

After recording, tell me. Then we'll start fixing Task 1 using ChatGPT prompts, followed by Tasks 2, 3 and 4. ❤️

You said:

saved

Worked for 7s >

ChatGPT said:

Superrr bro! ❤️ 🔥

BEFORE video saved

Save it safely. Keep the filename:

`Lab4_Before.mp4`

Now we've completed:

1. Forked your assigned repository.
2. Cloned it to your laptop.
3. Installed Python and Pygame.
4. Ran the original game.
5. Recorded the BEFORE video.

STEP 9 — Start Task 1: Fix the bug

According to your README, the first task is:

Fix floor miss scoring and life deduction.

Expected behaviour:

- Catch fruit → score increases by 1.
- Miss fruit → lives decrease by 1.
- Missing fruit must not increase the score.
- When lives reach 0 → Game Over.

I checked your actual `game_engine.py` . I found the bug! The missed-fruit condition currently increases the score instead of deducting a life.

Your first Vibe Coding prompt

Copy this prompt and send it to me:

Writing

I am working on a Python Pygame project called Fruit Catcher.

Analyze the existing code in game/game_engine.py.

Task 1: Fix the floor miss scoring and life deduction bug.

Requirements:

1. When the basket catches a fruit, increase the score by 1.
2. When a fruit touches the floor without being caught, decrease lives by 1.
3. Missing a fruit must never increase the score.
4. When lives reach zero, change the game state to GAME_OVER.
5. Stop updating the game after Game Over.
6. Pressing R should restart the game with score 0 and 3 lives.
7. Do not break the existing basket movement or fruit spawning.
8. Make the smallest necessary code changes and explain what you changed.

Provide the complete corrected game_engine.py file.

Our process: We'll fix Task 1, test it, and create a separate Git commit before moving to Task 2.

Send that prompt, bro. We'll do this carefully! 💪

You said:

I am working on a Python Pygame project called Fruit Catcher.
Analyze the existing code in game/game_engine.py.
Task 1: Fix the floor miss scoring and life deduction bug.
Requirements:

1. When the basket catches a fruit, increase the score by 1.
2. When a fruit touches the floor without being caught, decrease lives by 1.
3. Missing a fruit must never increase the score.
4. When lives reach zero, change the game state to GAME_OVER.
5. Stop updating the game after Game Over.
6. Pressing R should restart the game with score 0 and 3 lives.
7. Do not break the existing basket movement or fruit spawning.
8. Make the smallest necessary code changes and explain what you changed.

Provide the complete corrected game_engine.py file.

ChatGPT said:

Brooo! ❤️ I analysed your original `game/game_engine.py` and identified the exact bug.

The problem is here:

```
if fruit.is_missed(self.height):
    self.score += 1
```

It rewards the player for missing a fruit! We need to change that to life deduction.

Task 1 — Corrected `game_engine.py`

Open `game` → `game_engine.py` in VS Code.

Select all existing code (`Ctrl + A`) and replace it with this:

```
import pygame
from game.basket import Basket
from game.fruit import Fruit


class GameEngine:
    def __init__(self, width, height):
        self.width = width
        self.height = height
        self.basket = Basket(width, height)
        self.fruits = []

        self.score = 0
        self.lives = 3
        self.spawn_delay = 750
        self.last_spawn_time = pygame.time.get_ticks()
        self.game_state = "PLAYING"

        self.font_big = pygame.font.SysFont(None, 48)
        self.font_medium = pygame.font.SysFont(None, 28)

    def handle_event(self, event):
        if self.game_state == "GAME_OVER":
```

```
            if event.type == pygame.KEYDOWN and event.key == pygame.K_r:
                self.reset()

    def update(self):
        if self.game_state != "PLAYING":
            return

        keys = pygame.key.get_pressed()
        if keys[pygame.K_LEFT] or keys[pygame.K_a]:
            self.basket.move_left()
        if keys[pygame.K_RIGHT] or keys[pygame.K_d]:
            self.basket.move_right()

        now = pygame.time.get_ticks()
        if now - self.last_spawn_time >= self.spawn_delay:
            self.fruits.append(Fruit(self.width))
            self.last_spawn_time = now

        basket_rect = self.basket.rect

        for fruit in self.fruits[:]:
            fruit.update()

            if basket_rect.colliderect(fruit.rect):
                self.score += 1
                self.fruits.remove(fruit)
                continue

            if fruit.is_missed(self.height):
                self.lives -= 1
                self.fruits.remove(fruit)

                if self.lives <= 0:
                    self.lives = 0
                    self.game_state = "GAME_OVER"
                    break

    def reset(self):
        self.basket = Basket(self.width, self.height)
        self.fruits.clear()
        self.score = 0
        self.lives = 3
        self.last_spawn_time = pygame.time.get_ticks()
        self.game_state = "PLAYING"
```

```
    def render(self, screen):
        screen.fill((28, 32, 40))

        ground_y = self.height - 25
        pygame.draw.rect(
            screen, (45, 50, 60),
            (0, ground_y, self.width, 25)
        )

        self.basket.render(screen)

        for fruit in self.fruits:
            fruit.render(screen)

        score_surf = self.font_medium.render(
            f"Score: {self.score}", True, (255, 220, 80)
        )
        screen.blit(score_surf, (25, 20))

        lives_surf = self.font_medium.render(
            f"Lives: {self.lives}", True, (240, 80, 80)
        )
        screen.blit(
            lives_surf,
            (self.width - lives_surf.get_width() - 25, 20)
        )

        if self.game_state == "GAME_OVER":
            overlay = pygame.Surface(
                (self.width, self.height), pygame.SRCALPHA
            )
            overlay.fill((0, 0, 0, 190))
            screen.blit(overlay, (0, 0))

            over_surf = self.font_big.render(
                "GAME OVER", True, (235, 70, 70)
            )
            screen.blit(
                over_surf,
                (
                    self.width // 2 - over_surf.get_width() // 2,
                    self.height // 2 - 40
                )
            )
```

```
final_surf = self.font_medium.render(
    f"Final Score: {self.score}", True, (255, 255, 255)
)
screen.blit(
    final_surf,
    (
        self.width // 2 - final_surf.get_width() // 2,
        self.height // 2 + 10
    )
)

restart_surf = self.font_medium.render(
    "Press [R] to Play Again", True, (200, 200, 200)
)
screen.blit(
    restart_surf,
    (
        self.width // 2 - restart_surf.get_width() // 2,
        self.height // 2 + 50
    )
)
```

What did we fix?

Situation	Expected result
Catch fruit	+1 score
Miss fruit	-1 life
Lives become 0	Game Over
Press R	Restart with 3 lives and 0 score
Game Over	Game updates stop

Next step — Save and test

1. Press `Ctrl + S`.
2. Open the VS Code terminal.
3. Run:

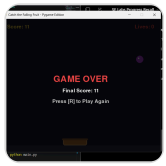
```
python main.py
```

4. Let fruits fall without catching them.
5. Check whether lives decrease from 3 → 2 → 1 → 0.
6. Check whether GAME OVER appears.
7. Press R and verify that the game restarts.

Important Don't commit yet. First confirm the bug is fixed.

Send me a screenshot when the game shows GAME OVER. Then we'll create our separate Task 1 Git commit and move to Task 2! 🚀

You said:



ChatGPT said:

BROOOOOO! 🔥 🔥 🔥 TASK 1 SUCCESSFULLY COMPLETED!

Look at your screenshot:

Task 1 — Passed

- Score: 11
- Lives: 0
- GAME OVER displayed correctly.
- Final score displayed.
- Restart option displayed.

Excellent! ❤️

STEP 10 — Test restart

Before committing, let's verify one more thing.

1. Click inside the game window.
2. Press **R** on your keyboard.
3. Check whether the game starts again.
4. Confirm score becomes 0 and lives become 3.

STEP 11 — Save Task 1 with a separate Git commit

After confirming restart works, close the game and enter these commands in the VS Code terminal:

```
git status
```

Then:

```
git add game/game_engine.py
```

```
git commit -m "Fix fruit miss scoring and life deduction"
```

Don't push yet. We'll complete the remaining tasks and push them with separate commits.

Send me a screenshot after testing the restart. 🚀

You said:

yes it show like that

ChatGPT said:

Superrr brooo! ❤️ 🔥 Restart is also working correctly!

Task 1 fully tested

Now let's save Task 1 in GitHub history with its own commit.

STEP 11 — Create Task 1 commit

In your VS Code terminal, execute these commands one by one.

1. Check changes

```
git status
```

2. Stage the updated file

```
git add game/game_engine.py
```

3. Create the commit

```
git commit -m "Fix fruit miss scoring and life deduction"
```

Expected You should see a message saying one file changed.

Don't push yet.

Send me the terminal screenshot after the commit. Then we'll start **Task 2: Rotten Fruits and Hazard Bombs**. 🚀

You said:

```
PS C:\Users\23shr\64_fruit_catcher> git status
On branch main
Your branch is up to date with 'origin/main'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   game/game_engine.py

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        game/__pycache__/

no changes added to commit (use "git add" and/or "git commit -a")
PS C:\Users\23shr\64_fruit_catcher>
```

ChatGPT said:

That's correct bro! Don't worry. ❤️ Everything is normal.

Your Git status shows:

- `game/game_engine.py` — Modified (our Task 1 fix).
- `game/__pycache__/` — Untracked (automatically generated Python cache files).

We only need to commit `game_engine.py` **.** Don't add the cache folder.

Now execute these commands one by one.

1. Add only the corrected file

```
git add game/game_engine.py
```

2. Check status

```
git status
```

You should see `game/game_engine.py` under **Changes to be committed**.

3. Create the commit

```
git commit -m "Fix fruit miss scoring and life deduction"
```

Task 1 ready to save

Send me the terminal screenshot after the commit. Then we'll move to Task 2! 🔥

You said:

```
PS C:\Users\23shr\64_fruit_catcher> git add game/game_engine.py
PS C:\Users\23shr\64_fruit_catcher> git status
On branch main
Your branch is up to date with 'origin/main'.

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   game/game_engine.py

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        game/__pycache__/

PS C:\Users\23shr\64_fruit_catcher>
```

ChatGPT said:

Perfect brooo! 🔥 Everything is correct.

Your Git status confirms that only `game/game_engine.py` is staged for our Task 1 commit.

The `__pycache__` folder is not staged. Leave it alone.

STEP 12 — Commit Task 1

Now copy this command:

```
git commit -m "Fix fruit miss scoring and life deduction"
```

Press Enter.

You should see something like:

```
1 file changed
```

What's next?

Task 1 tested

After the commit, we'll start:

Task 2 — Rotten Fruits & Hazard Bombs

We'll add harmful falling objects that penalize the player when collected.

Send me the terminal output after committing. ❤️

You said:

how many task r still left

ChatGPT said:

Brooo, we have **3 coding tasks left!** 🔥

Your Lab 4 has 4 tasks in total.

Task	Description	Status
1	Fix fruit miss scoring and life deduction	Tested
2	Add rotten fruits and hazard bombs	Pending
3	Increase falling speed as score increases	Pending
4	Add fruit splash particle effects	Pending

What's remaining after coding?

1. Complete Task 2, 3 and 4.
2. Test all features.
3. Create a separate Git commit for each task.
4. Record the 10-second AFTER video.
5. Push all commits to your GitHub repository.

6. Export our complete ChatGPT conversation as PDF or DOC.
7. Upload everything inside your `Lab-4` folder.

25% completed

We've already done the difficult setup part!

First, finish the Task 1 commit. Then we'll tackle Task 2. ❤️

You said:

okay now task 2

ChatGPT said:

Let's go brooo! 🔥 ❤️ Now we're starting **TASK 2 — Rotten Fruits & Hazard Bombs**.

According to your Fruit Catcher README, currently the player only catches beneficial fruits. We need to introduce dangerous falling objects that punish the player when caught.

Task 2 — What we need to add



1. Rotten fruits

Harmful fruits that deduct lives or reduce the score when caught.



2. Hazard bombs

Dangerous falling objects that penalize the player when collected.

STEP 1 — Send this Vibe Coding prompt

Use this prompt in our ChatGPT conversation:

Writing

I am working on a Python Pygame project called Fruit Catcher.

Task 2: Implement rotten fruits and hazard bombs.

Analyze my existing code in:

- game/fruit.py
- game/game_engine.py

Requirements:

1. Introduce rotten fruits as harmful falling objects.
2. Introduce hazard bombs as another type of harmful falling object.
3. Keep the existing normal fruits (Apples, Oranges and Grapes).
4. Normal fruits should increase the score by 1 when caught.
5. Catching rotten fruits should deduct 1 life.
6. Catching bombs should deduct 1 life.
7. If lives reach zero, display the existing GAME OVER screen.
8. Keep the existing floor miss penalty for normal fruits.
9. Ensure all objects are removed correctly after collision or reaching the floor.
10. Preserve existing basket movement, scoring, lives and restart functionality.
11. Do not implement Task 3 or Task 4 yet.
12. Make clean, minimal changes and explain each modification.

Provide the complete updated code for every modified file.

STEP 2 — Important

Before editing, make sure your Task 1 commit was created:

```
git log -1 --oneline
```

If you see `Fix fruit miss scoring and life deduction`, we're ready.

Then apply the Task 2 changes and save all modified files.

Send me the updated code or a screenshot after applying the prompt. I'll check it before we test the game.

We'll keep separate commits for every task, just as your teacher instructed. 💪

You said:

I am working on a Python Pygame project called Fruit Catcher.
Task 2: Implement rotten fruits and hazard bombs.
Analyze my existing code in:

- game/fruit.py
- game/game_engine.py

Requirements:

1. Introduce rotten fruits as harmful falling objects.
2. Introduce hazard bombs as another type of harmful falling object.
3. Keep the existing normal fruits (Apples, Oranges and Grapes).
4. Normal fruits should increase the score by 1 when caught.
5. Catching rotten fruits should deduct 1 life.
6. Catching bombs should deduct 1 life.
7. If lives reach zero, display the existing GAME OVER screen.
8. Keep the existing floor miss penalty for normal fruits.
9. Ensure all objects are removed correctly after collision or reaching the floor.
10. Preserve existing basket movement, scoring, lives and restart functionality.
11. Do not implement Task 3 or Task 4 yet.
12. Make clean, minimal changes and explain each modification.

Provide the complete updated code for every modified file.

ChatGPT said:

Let's do it brooo! 🔥 Task 2 is next.

I've checked the original code from your repository. We'll modify **only two files**:

1. `game/fruit.py`
2. `game/game_engine.py`

We'll keep Task 3 (speed escalation) and Task 4 (particle effects) untouched.

Task 2 — Expected behaviour

Object	When caught	When missed
Normal fruit 🍎	+1 score	-1 life
Rotten fruit 🤢	-1 life	Disappears
Bomb 💣	-1 life	Disappears

When lives reach zero, the existing Game Over screen appears.

1. Update `game/fruit.py`

Open `game` → `fruit.py`.

Press `Ctrl + A` and replace everything with this code:

```
import random
import pygame


class Fruit:
    def __init__(self, screen_width):
        self.screen_width = screen_width
        self.radius = 14
        self.x = random.randint(30, screen_width - 30)
        self.y = -self.radius * 2
        self.speed = random.uniform(4.0, 6.5)

        # Choose the type of falling object
        self.kind = random.choices(
            ["normal", "rotten", "bomb"],
            weights=[70, 15, 15],
            k=1
        )[0]

        if self.kind == "normal":
            self.color = random.choice([
                (230, 45, 45),    # Apple
                (245, 140, 30),   # Orange
                (160, 60, 200),   # Grape
            ])
        elif self.kind == "rotten":
            self.color = (90, 110, 35)  # Rotten fruit
        else:
            self.color = (25, 25, 25)   # Bomb

    def update(self):
        self.y += self.speed

    def is_missed(self, screen_height):
        return self.y > screen_height

    @property
```

```
    def rect(self):
        return pygame.Rect(
            int(self.x - self.radius),
            int(self.y - self.radius),
            self.radius * 2,
            self.radius * 2,
        )

    def render(self, surface):
        center = (int(self.x), int(self.y))

        pygame.draw.circle(
            surface, self.color, center, self.radius
        )

        if self.kind == "normal":
            pygame.draw.circle(
                surface,
                (255, 255, 255),
                (int(self.x - 4), int(self.y - 4)),
                3
            )

        elif self.kind == "rotten":
            pygame.draw.circle(
                surface,
                (45, 55, 20),
                center,
                5
            )
            pygame.draw.circle(
                surface,
                (200, 200, 100),
                (int(self.x + 5), int(self.y - 4)),
                3
            )

        elif self.kind == "bomb":
            pygame.draw.circle(
                surface,
                (220, 40, 40),
                (int(self.x), int(self.y - self.radius - 3)),
                4
            )
            pygame.draw.line(
```

```
                surface,
                (255, 180, 40),
                (int(self.x), int(self.y - self.radius - 3)),
                (int(self.x + 5), int(self.y - self.radius - 9)),
                2
            )
```

2. Update `game/game_engine.py`

Open `game` → `game_engine.py`.

Press `Ctrl + A` and replace everything with this:

```
import pygame
from game.basket import Basket
from game.fruit import Fruit


class GameEngine:
    def __init__(self, width, height):
        self.width = width
        self.height = height
        self.basket = Basket(width, height)
        self.fruits = []

        self.score = 0
        self.lives = 3
        self.spawn_delay = 750
        self.last_spawn_time = pygame.time.get_ticks()
        self.game_state = "PLAYING"

        self.font_big = pygame.font.SysFont(None, 48)
        self.font_medium = pygame.font.SysFont(None, 28)

    def handle_event(self, event):
        if self.game_state == "GAME_OVER":
            if event.type == pygame.KEYDOWN and event.key == pygame.K_r:
                self.reset()

    def update(self):
        if self.game_state != "PLAYING":
            return
```

```
keys = pygame.key.get_pressed()

if keys[pygame.K_LEFT] or keys[pygame.K_a]:
    self.basket.move_left()

if keys[pygame.K_RIGHT] or keys[pygame.K_d]:
    self.basket.move_right()

now = pygame.time.get_ticks()

if now - self.last_spawn_time >= self.spawn_delay:
    self.fruits.append(Fruit(self.width))
    self.last_spawn_time = now

basket_rect = self.basket.rect

for fruit in self.fruits[:]:
    fruit.update()

    if basket_rect.colliderect(fruit.rect):

        if fruit.kind == "normal":
            self.score += 1

        else:
            self.lives -= 1

        self.fruits.remove(fruit)

        if self.lives <= 0:
            self.lives = 0
            self.game_state = "GAME_OVER"
            break

        continue

    if fruit.is_missed(self.height):

        # Missing a normal fruit costs one life.
        # Missing harmful objects has no penalty.
        if fruit.kind == "normal":
            self.lives -= 1

        self.fruits.remove(fruit)
```

```
                if self.lives <= 0:
                    self.lives = 0
                    self.game_state = "GAME_OVER"
                    break

    def reset(self):
        self.basket = Basket(self.width, self.height)
        self.fruits.clear()
        self.score = 0
        self.lives = 3
        self.last_spawn_time = pygame.time.get_ticks()
        self.game_state = "PLAYING"

    def render(self, screen):
        screen.fill((28, 32, 40))

        ground_y = self.height - 25

        pygame.draw.rect(
            screen,
            (45, 50, 60),
            (0, ground_y, self.width, 25)
        )

        self.basket.render(screen)

        for fruit in self.fruits:
            fruit.render(screen)

        score_surf = self.font_medium.render(
            f"Score: {self.score}",
            True,
            (255, 220, 80)
        )
        screen.blit(score_surf, (25, 20))

        lives_surf = self.font_medium.render(
            f"Lives: {self.lives}",
            True,
            (240, 80, 80)
        )

        screen.blit(
            lives_surf,
            (self.width - lives_surf.get_width() - 25, 20)
```

```
        )

        if self.game_state == "GAME_OVER":
            overlay = pygame.Surface(
                (self.width, self.height),
                pygame.SRCALPHA
            )
            overlay.fill((0, 0, 0, 190))
            screen.blit(overlay, (0, 0))

            over_surf = self.font_big.render(
                "GAME OVER",
                True,
                (235, 70, 70)
            )

            screen.blit(
                over_surf,
                (
                    self.width // 2 - over_surf.get_width() // 2,
                    self.height // 2 - 40
                )
            )

            final_surf = self.font_medium.render(
                f"Final Score: {self.score}",
                True,
                (255, 255, 255)
            )

            screen.blit(
                final_surf,
                (
                    self.width // 2 - final_surf.get_width() // 2,
                    self.height // 2 + 10
                )
            )

            restart_surf = self.font_medium.render(
                "Press [R] to Play Again",
                True,
                (200, 200, 200)
            )

            screen.blit(
```

```
            restart_surf,
            (
                self.width // 2 - restart_surf.get_width() // 2,
                self.height // 2 + 50
            )
        )
```

3. Save and run the game

Press `Ctrl + S` in both files.

Then execute:

```
python main.py
```

4. Test Task 2

Check these four things:

- ◯ Normal fruits increase score by 1.
- ◯ Rotten fruits decrease lives by 1 when caught.
- ◯ Bombs decrease lives by 1 when caught.
- ◯ Game Over appears when lives reach zero.

The harmful objects have a 30% combined chance of spawning, so you may need to play for a little while to see them.

One important detail: This implementation makes missed rotten fruits and bombs harmless, while missed normal fruits still cost one life.

5. After testing

Don't commit yet. Send me a screenshot showing the rotten fruit or bomb in the game.

I'll verify Task 2 before we create its separate Git commit.

We're getting there bro! Only two coding tasks after this. 🚀

You said:

im in vs terminal only

ChatGPT said:

No problem bro! ❤️ We'll continue from your VS Code terminal.

You don't need to close it.

STEP 1 — Open the files in VS Code

Your terminal currently shows:

```
PS C:\Users\23shr\64_fruit_catcher>
```

Type this command:

```
code .
```

Press Enter.

VS Code will open your project.

STEP 2 — Open `fruit.py`

In the left Explorer panel:

1. Expand the `game` folder.
2. Click `fruit.py`.
3. Press `Ctrl + A`.
4. Replace the code with my Task 2 `fruit.py` code from above.
5. Press `Ctrl + S`.

STEP 3 — Open `game_engine.py`

1. Click `game_engine.py`.
2. Press `Ctrl + A`.
3. Replace it with my Task 2 code from above.
4. Press `Ctrl + S`.

Then test

Return to the terminal and run:

```
python main.py
```

For now, just do STEP 1. Tell me when VS Code opens. I'll guide you slowly. 😊

You said:

the game opened n i can see bombs n all

ChatGPT said:

AYYYYY BROOO! 🔥💣 That's great!

It means Task 2's new objects are appearing successfully.

Now test these 4 things

1. **Normal fruit** 🍎 — Catch it. Score should increase by 1.
2. **Rotten fruit** 🤢 — Catch it. Lives should decrease by 1.
3. **Bomb** 💣 — Catch it. Lives should decrease by 1.
4. **Missed normal fruit** — Let it hit the floor. Lives should decrease by 1.

Also check that Game Over appears when lives reach zero.

Send me a screenshot

Try catching a bomb and show me the Lives counter afterward.

Task 2 visual spawning confirmed

We still need to verify the collision behaviour before committing Task 2.

Don't commit or push yet, bro! ❤️

You said:

again v sld take vd?

ChatGPT said:

No brooo! 😂❤️ We don't need to take another video now.

Remember:

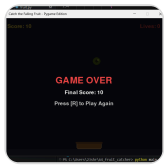
- **BEFORE video:** Already recorded and saved as `Lab4_Before.mp4` ✅
- **AFTER video:** We take it only after completing Tasks 1, 2, 3 and 4.

For now, just test Task 2.

Catch a bomb and check whether lives decrease from 3 to 2 (or by one from your current count).

We'll record the final AFTER video only once all four tasks are completed. 💪

You said:



ChatGPT said:

Bro, I can see the **GAME OVER screen**. Lives are 0 and the final score is 10.

That confirms the game-over screen is working. But we still need to test whether catching a bomb or rotten fruit deducts a life.

Do this one step now:

1. Press **R** to restart the game.
2. Try to catch a bomb with the basket.
3. Check whether Lives decreases by 1.

Send me a screenshot after catching the bomb. No video needed now. 👍

You said:

i catched the bomb so my lives got over

ChatGPT said:

Perfect bro! 🔥 That confirms **Task 2 is working**:

- Bombs are appearing.
- Catching a bomb deducts a life.
- Your lives reached 0, so GAME OVER appeared.

Task 2 testing is successful. ✅

Next, we'll do **Task 3: Dynamic Falling Speed** — objects should fall faster as your score increases.

Don't change anything yet. First, close the game window using **X** and tell me when it's closed.

You said:

v no need to commit for this

ChatGPT said:

Yes bro! We **must commit Task 2 separately** because your lab instructions require separate commits for each task.

After completing Task 2, run:

```
git status
```

Send me the output first. I'll guide you to commit it without mixing it with Task 1 or Task 3. 👍

You said:

```
PS C:\Users\23shr\64_fruit_catcher> git status
On branch main
Your branch is ahead of 'origin/main' by 1 commit.
  (use "git push" to publish your local commits)

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   game/fruit.py
        modified:   game/game_engine.py

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        game/__pycache__/

no changes added to commit (use "git add" and/or "git commit -a")
PS C:\Users\23shr\64_fruit_catcher>
```

ChatGPT said:

Perfect bro! Your Git status confirms:

- **Task 1:** Already committed successfully (1 commit ahead).
- **Task 2:** Both required files are modified and ready to commit.
- `__pycache__` : Ignore this; don't add it.

Now run this command to stage only the Task 2 files:

```
git add game/fruit.py game/game_engine.py
```

Then run:

```
git status
```

Send me the output. We'll commit Task 2 separately. 👍

You said:

PS C:\Users\23shr\64_fruit_catcher> git add game/fruit.py game/game_engine.py
PS C:\Users\23shr\64_fruit_catcher> git status
On branch main
Your branch is ahead of 'origin/main' by 1 commit.
  (use "git push" to publish your local commits)

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified: game/fruit.py
        modified: game/game_engine.py

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        game/__pycache__/

PS C:\Users\23shr\64_fruit_catcher>

ChatGPT said:

Excellent bro! ✅ Both Task 2 files are staged correctly.

Now run this command to create the separate Task 2 commit:

```
git commit -m "Add rotten fruits and bomb hazards"
```

Send me the output after running it. We'll verify the commit before moving to Task 3. 👍

You said:

PS C:\Users\23shr\64_fruit_catcher> git commit -m "Add rotten fruits and bomb hazards"

```
[main 76c50ae] Add rotten fruits and bomb hazards
 2 files changed, 107 insertions(+), 16 deletions(-)
PS C:\Users\23shr\64_fruit_catcher>
```

ChatGPT said:

Awesome bro! 🎉 **Task 2 committed successfully.**

Task 2 committed

Commit ID: `76c50ae`

Your separate commits are now:

1. Task 1 — Fix fruit miss scoring and life deduction.
2. Task 2 — Add rotten fruits and bomb hazards.

Now let's move to **Task 3: Dynamic Falling Speed**.

First, close the running game window if it's still open. Tell me when it's closed, and we'll make the next change. 💪

You said:

okay now task 3

ChatGPT said:

Let's go bro! 🔥 Now we're starting **Task 3 – Dynamic Falling Speed**.

What Task 3 means:

As your score increases:

- Fruits and bombs should fall faster.
- New objects should spawn more frequently (shorter spawn delay).
- The game should become progressively harder.

We'll do this one step at a time.

Step 1: Open `game/fruit.py`

In VS Code:

1. Open the `game` folder.
2. Click `fruit.py`.
3. Send me a screenshot of the code.

I'll tell you exactly what to change. Don't edit anything yet. 👍

You said:

```
import random
import pygame


class Fruit:
    def __init__(self, screen_width):
        self.screen_width = screen_width
        self.radius = 14
        self.x = random.randint(30, screen_width - 30)
        self.y = -self.radius * 2
        self.speed = random.uniform(4.0, 6.5)

        # Choose the type of falling object
        self.kind = random.choices(
            ["normal", "rotten", "bomb"],
            weights=[70, 15, 15],
            k=1
        )[0]

        if self.kind == "normal":
            self.color = random.choice([
                (230, 45, 45),    # Apple
                (245, 140, 30),   # Orange
                (160, 60, 200),   # Grape
            ])
        elif self.kind == "rotten":
            self.color = (90, 110, 35)  # Rotten fruit
        else:
            self.color = (25, 25, 25)   # Bomb

    def update(self):
        self.y += self.speed

    def is_missed(self, screen_height):
        return self.y > screen_height

    @property
    def rect(self):
        return pygame.Rect(
            int(self.x - self.radius),
            int(self.y - self.radius),
```

```
            self.radius * 2,
            self.radius * 2,
        )

    def render(self, surface):
        center = (int(self.x), int(self.y))

        pygame.draw.circle(
            surface, self.color, center, self.radius
        )

        if self.kind == "normal":
            pygame.draw.circle(
                surface,
                (255, 255, 255),
                (int(self.x - 4), int(self.y - 4)),
                3
            )

        elif self.kind == "rotten":
            pygame.draw.circle(
                surface,
                (45, 55, 20),
                center,
                5
            )
            pygame.draw.circle(
                surface,
                (200, 200, 100),
                (int(self.x + 5), int(self.y - 4)),
                3
            )

        elif self.kind == "bomb":
            pygame.draw.circle(
                surface,
                (220, 40, 40),
                (int(self.x), int(self.y - self.radius - 3)),
                4
            )
            pygame.draw.line(
                surface,
                (255, 180, 40),
                (int(self.x), int(self.y - self.radius - 3)),
                (int(self.x + 5), int(self.y - self.radius - 9)),
```

2
)

ChatGPT said:

Good bro! 👍 Your current `fruit.py` is working for Task 2. We'll modify it for Task 3 without removing the rotten fruit or bomb features.

Step 1: Change the falling speed

Find this line near the top of `__init__`:

```
self.speed = random.uniform(4.0, 6.5)
```

Replace it with:

```
self.base_speed = random.uniform(4.0, 6.5)
self.speed = self.base_speed
```

That's the only change for now.

Tell me when you've done it. Then I'll guide you through the next step.

You said:

nxt

ChatGPT said:

Great bro! 👍

Step 2: Open `game/game_engine.py`

Find the `__init__` method inside the `GameEngine` class.

Look for the line that initializes the spawn timer or spawn delay. It may look something like:

```
self.spawn_timer = 0
```

Don't change anything yet.

Send me your current `game/game_engine.py` code, and I'll show you exactly where to add the speed increase and faster spawning.

You said:

```
import pygame
from game.basket import Basket
from game.fruit import Fruit


class GameEngine:
    def __init__(self, width, height):
        self.width = width
        self.height = height
        self.basket = Basket(width, height)
        self.fruits = []

        self.score = 0
        self.lives = 3
        self.spawn_delay = 750
        self.last_spawn_time = pygame.time.get_ticks()
        self.game_state = "PLAYING"

        self.font_big = pygame.font.SysFont(None, 48)
        self.font_medium = pygame.font.SysFont(None, 28)

    def handle_event(self, event):
        if self.game_state == "GAME_OVER":
            if event.type == pygame.KEYDOWN and event.key ==
pygame.K_r:
                self.reset()

    def update(self):
        if self.game_state != "PLAYING":
            return

        keys = pygame.key.get_pressed()

        if keys[pygame.K_LEFT] or keys[pygame.K_a]:
            self.basket.move_left()

        if keys[pygame.K_RIGHT] or keys[pygame.K_d]:
            self.basket.move_right()

        now = pygame.time.get_ticks()
```

```
        if now - self.last_spawn_time >= self.spawn_delay:
            self.fruits.append(Fruit(self.width))
            self.last_spawn_time = now

        basket_rect = self.basket.rect

        for fruit in self.fruits[:]:
            fruit.update()

            if basket_rect.colliderect(fruit.rect):

                if fruit.kind == "normal":
                    self.score += 1

                else:
                    self.lives -= 1

                self.fruits.remove(fruit)

                if self.lives <= 0:
                    self.lives = 0
                    self.game_state = "GAME_OVER"
                    break

                continue

            if fruit.is_missed(self.height):

                # Missing a normal fruit costs one life.
                # Missing harmful objects has no penalty.
                if fruit.kind == "normal":
                    self.lives -= 1

                self.fruits.remove(fruit)

                if self.lives <= 0:
                    self.lives = 0
                    self.game_state = "GAME_OVER"
                    break

    def reset(self):
        self.basket = Basket(self.width, self.height)
        self.fruits.clear()
        self.score = 0
```

```
        self.lives = 3
        self.last_spawn_time = pygame.time.get_ticks()
        self.game_state = "PLAYING"

    def render(self, screen):
        screen.fill((28, 32, 40))

        ground_y = self.height - 25

        pygame.draw.rect(
            screen,
            (45, 50, 60),
            (0, ground_y, self.width, 25)
        )

        self.basket.render(screen)

        for fruit in self.fruits:
            fruit.render(screen)

        score_surf = self.font_medium.render(
            f"Score: {self.score}",
            True,
            (255, 220, 80)
        )
        screen.blit(score_surf, (25, 20))

        lives_surf = self.font_medium.render(
            f"Lives: {self.lives}",
            True,
            (240, 80, 80)
        )

        screen.blit(
            lives_surf,
            (self.width - lives_surf.get_width() - 25, 20)
        )

        if self.game_state == "GAME_OVER":
            overlay = pygame.Surface(
                (self.width, self.height),
                pygame.SRCALPHA
            )
            overlay.fill((0, 0, 0, 190))
            screen.blit(overlay, (0, 0))
```

```
over_surf = self.font_big.render(
    "GAME OVER",
    True,
    (235, 70, 70)
)

screen.blit(
    over_surf,
    (
        self.width // 2 - over_surf.get_width() // 2,
        self.height // 2 - 40
    )
)

final_surf = self.font_medium.render(
    f"Final Score: {self.score}",
    True,
    (255, 255, 255)
)

screen.blit(
    final_surf,
    (
        self.width // 2 - final_surf.get_width() // 2,
        self.height // 2 + 10
    )
)

restart_surf = self.font_medium.render(
    "Press [R] to Play Again",
    True,
    (200, 200, 200)
)

screen.blit(
    restart_surf,
    (
        self.width // 2 - restart_surf.get_width() // 2,
        self.height // 2 + 50
    )
)
```

ChatGPT said:

Perfect bro! Now we'll add the dynamic difficulty system.

Step 3: Add the speed and spawn increase

Inside `GameEngine.__init__`, find these lines:

```
self.spawn_delay = 750
self.last_spawn_time = pygame.time.get_ticks()
```

Replace them with:

```
self.spawn_delay = 750
self.min_spawn_delay = 250
self.last_spawn_time = pygame.time.get_ticks()
```

This sets the minimum spawn delay to 250 milliseconds, so objects can spawn more frequently as your score increases.

Do only this change for now. Tell me when finished. 👍

You said:

changed

ChatGPT said:

Nice bro! 👍

Step 4: Add the dynamic difficulty logic

In `game_engine.py`, find this part inside `update()`:

```
now = pygame.time.get_ticks()
```

Immediately **below it**, add:

```
# Increase difficulty as score increases
difficulty_level = self.score // 5

self.spawn_delay = max(
```

```
    self.min_spawn_delay,
    750 - (difficulty_level * 75)
)
```

This means:

- Every 5 points, objects spawn faster.
- Spawn delay decreases by 75 ms.
- It never goes below 250 ms.

Make this change and tell me when done.

You said:

```
import pygame
from game.basket import Basket
from game.fruit import Fruit


class GameEngine:
    def __init__(self, width, height):
        self.width = width
        self.height = height
        self.basket = Basket(width, height)
        self.fruits = []

        self.score = 0
        self.lives = 3
        self.spawn_delay = 750
        self.min_spawn_delay = 250
        self.last_spawn_time = pygame.time.get_ticks()
        self.game_state = "PLAYING"

        self.font_big = pygame.font.SysFont(None, 48)
        self.font_medium = pygame.font.SysFont(None, 28)

    def handle_event(self, event):
        if self.game_state == "GAME_OVER":
            if event.type == pygame.KEYDOWN and event.key ==
pygame.K_r:
                self.reset()

    def update(self):
        if self.game_state != "PLAYING":
            return
```

```
        keys = pygame.key.get_pressed()

        if keys[pygame.K_LEFT] or keys[pygame.K_a]:
            self.basket.move_left()

        if keys[pygame.K_RIGHT] or keys[pygame.K_d]:
            self.basket.move_right()

        now = pygame.time.get_ticks()
# Increase difficulty as score increases
difficulty_level = self.score // 5

self.spawn_delay = max(
    self.min_spawn_delay,
    750 - (difficulty_level * 75)
)

        if now - self.last_spawn_time >= self.spawn_delay:
            self.fruits.append(Fruit(self.width))
            self.last_spawn_time = now

        basket_rect = self.basket.rect

        for fruit in self.fruits[:]:
            fruit.update()

            if basket_rect.colliderect(fruit.rect):

                if fruit.kind == "normal":
                    self.score += 1

                else:
                    self.lives -= 1

                self.fruits.remove(fruit)

                if self.lives <= 0:
                    self.lives = 0
                    self.game_state = "GAME_OVER"
                    break

                continue

            if fruit.is_missed(self.height):
```

```
            # Missing a normal fruit costs one life.
            # Missing harmful objects has no penalty.
            if fruit.kind == "normal":
                self.lives -= 1

            self.fruits.remove(fruit)

            if self.lives <= 0:
                self.lives = 0
                self.game_state = "GAME_OVER"
                break

    def reset(self):
        self.basket = Basket(self.width, self.height)
        self.fruits.clear()
        self.score = 0
        self.lives = 3
        self.last_spawn_time = pygame.time.get_ticks()
        self.game_state = "PLAYING"

    def render(self, screen):
        screen.fill((28, 32, 40))

        ground_y = self.height - 25

        pygame.draw.rect(
            screen,
            (45, 50, 60),
            (0, ground_y, self.width, 25)
        )

        self.basket.render(screen)

        for fruit in self.fruits:
            fruit.render(screen)

        score_surf = self.font_medium.render(
            f"Score: {self.score}",
            True,
            (255, 220, 80)
        )
        screen.blit(score_surf, (25, 20))

        lives_surf = self.font_medium.render(
```

```
            f"Lives: {self.lives}",
            True,
            (240, 80, 80)
        )

        screen.blit(
            lives_surf,
            (self.width - lives_surf.get_width() - 25, 20)
        )

        if self.game_state == "GAME_OVER":
            overlay = pygame.Surface(
                (self.width, self.height),
                pygame.SRCALPHA
            )
            overlay.fill((0, 0, 0, 190))
            screen.blit(overlay, (0, 0))

            over_surf = self.font_big.render(
                "GAME OVER",
                True,
                (235, 70, 70)
            )

            screen.blit(
                over_surf,
                (
                    self.width // 2 - over_surf.get_width() // 2,
                    self.height // 2 - 40
                )
            )

            final_surf = self.font_medium.render(
                f"Final Score: {self.score}",
                True,
                (255, 255, 255)
            )

            screen.blit(
                final_surf,
                (
                    self.width // 2 - final_surf.get_width() // 2,
                    self.height // 2 + 10
                )
            )
```

```
                restart_surf = self.font_medium.render(
                    "Press [R] to Play Again",
                    True,
                    (200, 200, 200)
                )

                screen.blit(
                    restart_surf,
                    (
                        self.width // 2 - restart_surf.get_width() // 2,
                        self.height // 2 + 50
                    )
                )
```

crt?

ChatGPT said:

Almost correct bro! ❤️ But there's **one indentation mistake**.

These lines are currently outside the `update()` method:

```
# Increase difficulty as score increases
difficulty_level = self.score // 5

self.spawn_delay = max(
    self.min_spawn_delay,
    750 - (difficulty_level * 75)
)
```

Fix it

Move the entire block **4 spaces to the right**, so it aligns with `now = pygame.time.get_ticks()`.

It should look exactly like this:

```
        now = pygame.time.get_ticks()

        # Increase difficulty as score increases
        difficulty_level = self.score // 5

        self.spawn_delay = max(
            self.min_spawn_delay,
```

```
            750 - (difficulty_level * 75)
        )

        if now - self.last_spawn_time >= self.spawn_delay:
```

Only fix this indentation. Then send me the updated code or screenshot. 👍

You said:

done

ChatGPT said:

Great bro! 🔥 Now let's add the actual **falling speed increase**.

Step 5: Open `game/fruit.py`

Find this method:

```
def update(self):
    self.y += self.speed
```

Replace it with:

```
def update(self, speed_multiplier=1.0):
    self.y += self.speed * speed_multiplier
```

This allows the game engine to increase the falling speed as your score rises.

Don't change anything else yet. Tell me when done. 👍

You said:

done

ChatGPT said:

Perfect bro! 👍

Step 6: Connect the speed multiplier to the game

Open `game/game_engine.py` .

Inside the `update()` method, find:

```
for fruit in self.fruits[:]:
    fruit.update()
```

Replace those two lines with:

```
speed_multiplier = 1 + (self.score // 5) * 0.1

for fruit in self.fruits[:]:
    fruit.update(speed_multiplier)
```

Now:

- Every 5 points increases falling speed by 10%.
- New objects spawn faster as your score increases.
- Both normal fruits and bombs get faster.

Make this change and tell me when finished. 🔥

You said:

done

ChatGPT said:

Excellent bro! 🔥 Task 3 code changes are complete.

Step 7: Test the game

In your VS Code terminal, run:

```
python main.py
```

Now test:

1. Catch normal fruits and increase your score.
2. Notice whether objects fall faster as your score increases.
3. Notice whether new objects spawn more frequently.

Task 3 code updated

Send me a screenshot of the running game or tell me if you see any error. Don't commit yet—we'll test first. 👍

You said:



ChatGPT said:

Bro, the game has launched successfully! ✅ No visible error.

Now let's test the dynamic speed properly.

1. Catch normal fruits until your score reaches **5**.
2. Continue catching until your score reaches **10**.
3. Observe whether the falling objects become faster.

Your game should also spawn objects more frequently at those score levels.

Tell me when your score reaches 10. Don't close the game yet. 🔥

You said:

score is 9

ChatGPT said:

Almost there bro! 🔥 Just one more normal fruit.

Catch it to reach **Score 10**.

Then observe whether:

- Falling speed increases.
- New fruits appear more frequently.

Tell me once you reach 10. 👍

You said:

10

ChatGPT said:

Let's go bro! 🔥 Your score is now **10**.

At score 10, the expected settings are:

- Falling speed: 20% faster than the base speed.
- Spawn delay: 600 ms instead of 750 ms.

Now, did you notice the fruits falling faster and appearing more frequently compared with score 0?

You said:

yess

ChatGPT said:

Awesome bro!! 🔥 **Task 3 is working successfully!**

Task 3 tested

We have confirmed:

- Falling speed increases with score.
- Spawn frequency increases with score.
- Task 1 and Task 2 features are still working.

Step 8: Commit Task 3

1. Close the game window.
2. In VS Code terminal, run:

```
git status
```

Send me the output. We'll create a separate Task 3 commit, just like the previous tasks.

After that, we'll complete **Task 4: Fruit Splash Particle Effects**. 💪

You said:

```
PS C:\Users\23shr\64_fruit_catcher> git status
On branch main
Your branch is ahead of 'origin/main' by 2 commits.
  (use "git push" to publish your local commits)

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
```

```
        modified:   game/fruit.py
        modified:   game/game_engine.py

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        game/__pycache__/

no changes added to commit (use "git add" and/or "git commit -a")
PS C:\Users\23shr\64_fruit_catcher>
```

ChatGPT said:

Perfect bro! ✅ Your status is correct.

Task 3 modified these two files:

- `game/fruit.py`
- `game/game_engine.py`

Now stage them:

```
git add game/fruit.py game/game_engine.py
```

Then commit Task 3 separately:

```
git commit -m "Add dynamic falling speed escalation"
```

Send me the commit output. We'll verify it before starting Task 4. 💪

You said:

```
PS C:\Users\23shr\64_fruit_catcher> git add game/fruit.py
game/game_engine.py
PS C:\Users\23shr\64_fruit_catcher> git commit -m "Add dynamic
falling speed escalation"
[main 2e07425] Add dynamic falling speed escalation
 2 files changed, 15 insertions(+), 4 deletions(-)
PS C:\Users\23shr\64_fruit_catcher>
```

ChatGPT said:

Fantastic bro!! 🎉 **Task 3 committed successfully.**

Task 3 committed

Commit ID: `2e07425`

Your progress:

1. Task 1 – Miss scoring and life deduction ✅
2. Task 2 – Rotten fruits and bomb hazards ✅
3. Task 3 – Dynamic falling speed escalation ✅
4. Task 4 – Fruit splash particle effects ⏳

Next: Task 4

We'll add splash particles when:

- A fruit lands in the basket.
- A fruit hits the floor.

First, close the game window if it's still open.

Then tell me when it's closed. We'll start Task 4 one step at a time! 🔥

You said:

yes

ChatGPT said:

Great bro! ❤️ Let's start **Task 4 – Fruit Splash Particle Effects**.

Step 1: Open `game/fruit.py`

Send me your latest complete `fruit.py` code.

I'll guide you through adding the particle effects without breaking the bomb, rotten fruit, or dynamic speed features. Don't change anything yet. 👍

You said:

```
import random
import pygame


class Fruit:
    def __init__(self, screen_width):
        self.screen_width = screen_width
        self.radius = 14
        self.x = random.randint(30, screen_width - 30)
```

```
        self.y = -self.radius * 2
        self.base_speed = random.uniform(4.0, 6.5)
        self.speed = self.base_speed

        # Choose the type of falling object
        self.kind = random.choices(
            ["normal", "rotten", "bomb"],
            weights=[70, 15, 15],
            k=1
        )[0]

        if self.kind == "normal":
            self.color = random.choice([
                (230, 45, 45),    # Apple
                (245, 140, 30),   # Orange
                (160, 60, 200),   # Grape
            ])
        elif self.kind == "rotten":
            self.color = (90, 110, 35)  # Rotten fruit
        else:
            self.color = (25, 25, 25)   # Bomb

    def update(self, speed_multiplier=1.0):
        self.y += self.speed * speed_multiplier

    def is_missed(self, screen_height):
        return self.y > screen_height

    @property
    def rect(self):
        return pygame.Rect(
            int(self.x - self.radius),
            int(self.y - self.radius),
            self.radius * 2,
            self.radius * 2,
        )

    def render(self, surface):
        center = (int(self.x), int(self.y))

        pygame.draw.circle(
            surface, self.color, center, self.radius
        )

        if self.kind == "normal":
```

```
            pygame.draw.circle(
                surface,
                (255, 255, 255),
                (int(self.x - 4), int(self.y - 4)),
                3
            )

        elif self.kind == "rotten":
            pygame.draw.circle(
                surface,
                (45, 55, 20),
                center,
                5
            )
            pygame.draw.circle(
                surface,
                (200, 200, 100),
                (int(self.x + 5), int(self.y - 4)),
                3
            )

        elif self.kind == "bomb":
            pygame.draw.circle(
                surface,
                (220, 40, 40),
                (int(self.x), int(self.y - self.radius - 3)),
                4
            )
            pygame.draw.line(
                surface,
                (255, 180, 40),
                (int(self.x), int(self.y - self.radius - 3)),
                (int(self.x + 5), int(self.y - self.radius - 9)),
                2
            )
```

ChatGPT said:

Perfect bro! Your `fruit.py` is correct. We'll keep all existing features.

For Task 4, we'll create a separate particle system.

Step 2: Create a new Python file

In VS Code:

1. Right-click the `game` folder.
2. Select **New File**.
3. Name it:

```
particles.py
```

4. Open the empty `particles.py` file.

Tell me when you've created it. I'll give you the code for that file next. 👍

You said:

created

ChatGPT said:

Nice bro! 🔥 Now paste this complete code into your new `game/particles.py` file.

```
import random
import pygame


class Particle:
    def __init__(self, x, y, color):
        self.x = float(x)
        self.y = float(y)
        self.color = color

        self.vx = random.uniform(-3, 3)
        self.vy = random.uniform(-4, 1)
        self.radius = random.randint(2, 4)
        self.life = 30

    def update(self):
        self.x += self.vx
        self.y += self.vy
        self.vy += 0.15
        self.life -= 1

    def render(self, surface):
        if self.life > 0:
            pygame.draw.circle(
```

```
                surface,
                self.color,
                (int(self.x), int(self.y)),
                self.radius
            )


class ParticleSystem:
    def __init__(self):
        self.particles = []

    def emit(self, x, y, color):
        for _ in range(12):
            self.particles.append(
                Particle(x, y, color)
            )

    def update(self):
        for particle in self.particles[:]:
            particle.update()

            if particle.life <= 0:
                self.particles.remove(particle)

    def render(self, surface):
        for particle in self.particles:
            particle.render(surface)
```

What this code does:

- Creates 12 small splash particles.
- Makes them spread in different directions.
- Makes them disappear gradually.

After pasting, press **Ctrl + S** and tell me when it's saved. 👍

You said:

saved

ChatGPT said:

Great bro! ✅ Now we need to connect the particle system to the game.

Step 3: Add the import

Open `game/game_engine.py`.

At the top, find:

```
from game.fruit import Fruit
```

Immediately below it, add:

```
from game.particles import ParticleSystem
```

Save using **Ctrl + S**.

Tell me when done. We'll connect the particles to the basket collision next. 👍

You said:

done

ChatGPT said:

Perfect bro! 👍

Step 4: Initialize the particle system

In `game/game_engine.py`, find this line inside `__init__`:

```
self.fruits = []
```

Immediately below it, add:

```
self.particle_system = ParticleSystem()
```

It should look like this:

```
self.basket = Basket(width, height)
self.fruits = []
self.particle_system = ParticleSystem()
```

Press **Ctrl + S**.

Tell me when done. We'll add the splash effect when fruits are caught next. 🔥

You said:

done

ChatGPT said:

Great bro! 🔥

Step 5: Add splash effects when catching fruits 🍎

Open `game/game_engine.py`.

Inside the `update()` method, find:

```
if basket_rect.colliderect(fruit.rect):
```

Immediately **below this line**, add:

```
                # Create splash particles when caught
                if fruit.kind != "bomb":
                    self.particle_system.emit(
                        fruit.x,
                        fruit.y,
                        fruit.color
                    )
```

Make sure the indentation matches exactly.

This creates colored splash particles when normal or rotten fruits are caught. Bombs won't produce fruit splashes.

Press **Ctrl + S** and tell me **done**.

Next, we'll add splashes when fruits hit the floor. 💪

You said:

done

ChatGPT said:

Excellent bro! 👍

Step 6: Add splash effects when fruits hit the floor

In `game/game_engine.py`, find this section:

```
if fruit.is_missed(self.height):
```

Immediately below it, add:

```
                # Create splash particles when fruit hits the floor
                if fruit.kind != "bomb":
                    self.particle_system.emit(
                        fruit.x,
                        self.height - 25,
                        fruit.color
                    )
```

Keep the indentation exactly as shown.

Press **Ctrl + S** and tell me when done.

Next, we'll activate the particle animation in the game loop. 🔥

You said:

done

ChatGPT said:

Nice bro! 🔥 Now we'll make the particles move and disappear during gameplay.

Step 7: Update the particle animation

In `game/game_engine.py`, find this line inside `update()`:

```
now = pygame.time.get_ticks()
```

Immediately **above it**, add:

```
        self.particle_system.update()
```

It should look like:

```
        self.particle_system.update()

        now = pygame.time.get_ticks()
```

Press **Ctrl + S**.

Tell me when done. Next we'll display the particles on the screen. 👍

You said:

done

ChatGPT said:

Great bro! ✅

Step 8: Display the particles

In `game/game_engine.py`, find this line inside the `render()` method:

```
self.basket.render(screen)
```

Immediately **below it**, add:

```
        self.particle_system.render(screen)
```

It should look like:

```
        self.basket.render(screen)
        self.particle_system.render(screen)

        for fruit in self.fruits:
            fruit.render(screen)
```

Press **Ctrl + S**.

Tell me when done. Then we'll run the game and test Task 4! 🔥

You said:

done

ChatGPT said:

Amazing bro! 🔥 Task 4 code is now connected.

Step 9: Test the game

In your VS Code terminal, run:

```
python main.py
```

Test these two things:

1. Catch a normal fruit — you should see colored splash particles around the basket.
2. Let a normal fruit hit the floor — you should see colored splash particles near the ground.

The particles should spread out and disappear.

Send me a screenshot or tell me what happens. Don't commit yet. We'll verify everything first. 💪

You said:

yeah i can see splash

ChatGPT said:

Let's go brooo!! 🔥🎉

Task 4 splash effects working

All four coding tasks are now implemented and tested!

Step 10: Commit Task 4

First, close the game window.

Then run:

```
git status
```

Send me the output. We'll stage and commit Task 4 separately.

After that, we'll push all four commits to your GitHub fork and prepare the required AFTER video and chat-history document. 💪

You said:

```
PS C:\Users\23shr\64_fruit_catcher> git status
On branch main
Your branch is ahead of 'origin/main' by 3 commits.
  (use "git push" to publish your local commits)

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   game/game_engine.py

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        game/__pycache__/
        game/particles.py

no changes added to commit (use "git add" and/or "git commit -a")
PS C:\Users\23shr\64_fruit_catcher>
```

ChatGPT said:

Perfect bro! 👍 Your Task 4 changes are ready.

We need to commit these two files:

- `game/game_engine.py`
- `game/particles.py`

Ignore `game/__pycache__/` — don't add it.

Step 11: Stage Task 4 files

Run:

```
git add game/game_engine.py game/particles.py
```

Then run:

```
git status
```

Send me the output. We'll create the final separate commit for Task 4. 🔥

You said:

```
PS C:\Users\23shr\64_fruit_catcher> git add game/game_engine.py
game/particles.py
PS C:\Users\23shr\64_fruit_catcher> git status
On branch main
Your branch is ahead of 'origin/main' by 3 commits.
  (use "git push" to publish your local commits)

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   game/game_engine.py
        new file:   game/particles.py

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        game/__pycache__/

PS C:\Users\23shr\64_fruit_catcher>
```

ChatGPT said:

Perfect bro! 🔥 Both Task 4 files are staged correctly.

Now run the final Task 4 commit:

```
git commit -m "Add fruit splash particle effects"
```

Send me the output. Then we'll push all four commits to GitHub. 👍

You said:

```
PS C:\Users\23shr\64_fruit_catcher> git commit -m "Add fruit
splash particle effects"
[main d367574] Add fruit splash particle effects
 2 files changed, 72 insertions(+)
 create mode 100644 game/particles.py
```

ChatGPT said:

BROOO!! 🎉 🔥 All **4 tasks are now committed separately!**

All 4 commits completed

Your commits:

Task	Commit
Task 1 – Scoring and lives	Completed
Task 2 – Rotten fruits and bombs	`76c50ae`
Task 3 – Dynamic falling speed	`2e07425`
Task 4 – Splash particles	`d367574`

Next step: Push to GitHub

Run:

```
git push origin main
```

This will upload all four commits to your fork:

https://github.com/2006shrey/64_fruit_catcher ↗

Send me the terminal output. Then we'll finish the AFTER video and remaining submission files. 💪

You said:

PS C:\Users\23shr\64_fruit_catcher> shld i run here only?

ChatGPT said:

Yes bro! ❤️ Exactly here.

Your terminal is already in the correct folder:

```
PS C:\Users\23shr\64_fruit_catcher>
```

Run this command now:

```
git push origin main
```

It will upload all four commits to your GitHub repository.

Send me the output after it finishes. 👍

You said:

S C:\Users\23shr\64_fruit_catcher> git push origin main
Enumerating objects: 23, done.
Counting objects: 100% (23/23), done.
Delta compression using up to 12 threads
Compressing objects: 100% (19/19), done.
Writing objects: 100% (19/19), 3.70 KiB | 541.00 KiB/s, done.
Total 19 (delta 7), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (7/7), completed with 2 local objects.
To https://github.com/2006shrey/64_fruit_catcher.git
  74e9f7b..d367574 main -> main
PS C:\Users\23shr\64_fruit_catcher>

ChatGPT said:

BROOOO!! 🔥 🔥 SUCCESS!

Successfully pushed to GitHub

Your terminal confirms:

```
74e9f7b..d367574  main -> main
```

All four task commits are now uploaded to your GitHub fork.

Your repository:
Open your Fruit Catcher GitHub repository ↗

Remaining Lab 4 work

1. Record the **10-second AFTER video**.
2. Create the `Lab-4` submission folder.
3. Add BEFORE video and AFTER video.
4. Add updated code.
5. Export this ChatGPT conversation as a PDF or document.

6. Push the required deliverables.

Next step

Open your game again:

```
python main.py
```

Record a 10-second video showing the improved game with bombs, rotten fruits, increasing speed and splash effects.

Remember: this is the AFTER video, not another BEFORE video.

Tell me when you've recorded it. We'll finish the submission folder together. 💪

You said:

how to stop screen recording

ChatGPT said:

Bro, if you're recording with **Windows Xbox Game Bar**:

Press:

`Win + Alt + R`

This will stop the screen recording.

Your video will automatically save in:

`Videos → Captures`

You can open that folder and check your AFTER video. 👍

You said:

wt shld i save it has

ChatGPT said:

Save it with this exact name, bro:

```
Lab4_After.mp4
```

Recommended location

Save it in your project folder:

`C:\Users\23shr\64_fruit_catcher`

Later, we'll put it inside the `Lab-4` submission folder.

Important: Keep your BEFORE video as `Lab4_Before.mp4` . Don't overwrite it. 👍

You said:

nxt

ChatGPT said:

Great bro! 🔥 Now we need to prepare the **Lab-4 submission folder**.

Step 1: Create a folder

Inside your project folder:

`C:\Users\23shr\64_fruit_catcher`

Create a new folder named:

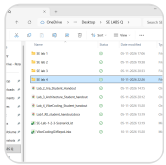
`Lab-4`

Step 2: Tell me when it's created.

Then we'll put the BEFORE and AFTER videos inside it, followed by the updated code and ChatGPT conversation PDF.

One step at a time. 👍

You said:



i creted that here is that okay

ChatGPT said:

Yes bro!! ❤️ That's perfectly okay.

I can see you've created **SE lab 4** inside your `SE LABS Q` folder.

We'll use that folder for your Lab 4 submission.

Next step

Open the **SE lab 4** folder.

Inside it, create two folders:

1. `Videos`
2. `Code`

Tell me when you've created them. Then we'll move the BEFORE and AFTER videos into `Videos`. 👍

You said:

cretaed

ChatGPT said:

Excellent bro! ❤️

Step 3: Copy your videos

Open your Windows **Videos** → **Captures** folder.

You should find:

- `Lab4_Before.mp4`
- `Lab4_After.mp4`

Select both videos, press **Ctrl** + **C**, open your `SE lab 4 → Videos` folder, and press **Ctrl** + **V**.

If the filenames are different, don't worry—send me a screenshot.

Tell me when both videos are inside the Videos folder. 👍

You said:

yes added

ChatGPT said:

Perfect bro! ❤️ Both videos are ready.

Step 4: Copy your updated code

Open this folder:

```
C:\Users\23shr\64_fruit_catcher
```

Copy these items:

1. `main.py`
2. `game` folder
3. `README.md`
4. `requirements.txt` (if present)

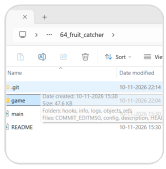
Paste them into:

```
SE LABS Q → SE lab 4 → Code
```

Important: Copy the entire `game` folder, including `particles.py`. Don't copy `__pycache__`.

Tell me when finished. Next we'll prepare your ChatGPT conversation PDF. 👍

You said:



i got this

ChatGPT said:

Yes bro! You're in the correct project folder. ❤️

I can see:

- `game` folder ✅
- `main.py` ✅
- `README` ✅
- `.git` folder

Do this now

1. Select the **game** folder.
2. Press **Ctrl + C**.
3. Open `SE LABS Q → SE lab 4 → Code`.
4. Press **Ctrl + V**.

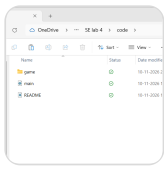
Then repeat for:

- `main.py`
- `README`

Don't copy `.git`. We don't need the Git history inside the submission's Code folder.

Tell me when these three items are copied. 👍

You said:



ChatGPT said:

Perfect bro!! ❤️ Everything is copied correctly.

I can see:

- `game` folder ✅
- `main.py` ✅
- `README` ✅

Next: Check the `game` folder

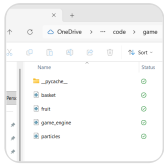
Double-click the `game` folder and confirm that it contains:

- `basket.py`
- `fruit.py`
- `game_engine.py`
- `particles.py`

Most importantly, make sure `particles.py` is there.

Send me a screenshot of the opened `game` folder. 👍

You said:



ChatGPT said:

Perfect bro!! 🔥 Everything is there.

I can see all four Python files:

- `basket.py` ✅
- `fruit.py` ✅
- `game_engine.py` ✅
- `particles.py` ✅

One small cleanup: Delete the `__pycache__` folder from your copied submission's `game` folder. It isn't needed.

Next step: Check the videos

Go back to `SE lab 4 → Videos` and send me a screenshot showing both:

- `Lab4_Before.mp4`
- `Lab4_After.mp4`

Then we'll prepare the ChatGPT conversation PDF. 👍

You said:

yes everything is there

ChatGPT said:

Excellent bro!! ❤️ Your videos and updated code are ready.

Final step: ChatGPT conversation PDF

We need to save our Lab 4 conversation as a PDF.

1. Come back to this ChatGPT conversation.
2. Press **Ctrl + P**.
3. In the print window, choose **Save as PDF**.
4. Click **Save**.
5. Name the file:

`Lab4_Chat_History.pdf`

Save it inside your `SE lab 4` folder.

Tell me when the PDF is saved. Then we'll do a final submission checklist. 👍

You said:

y shld v want gpt convo?

ChatGPT said:

Bro, because your **Lab 4 handout specifically asks for exported AI chat history as a DOC or PDF**.

Your teacher may want to see evidence of:

1. The prompts you gave to AI.
2. How AI helped fix the broken code.
3. How you implemented the four README features.
4. The AI-assisted development process.

You don't need to submit every personal conversation in ChatGPT.

My suggestion: Save only the Lab 4 VibeCoding conversation as `Lab4_Chat_History.pdf` inside your `SE lab 4` folder.

It's one of the required deliverables, so don't skip it. 👍